Week 1 exercises

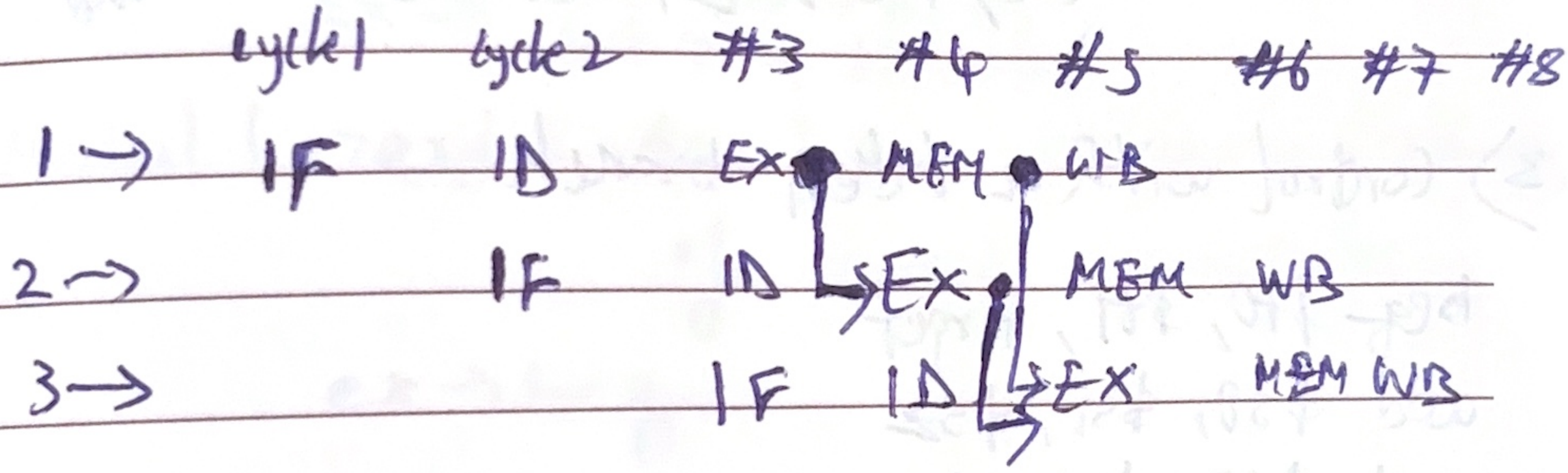
Part A:

1) Pure RAW

add $s0, $t0, $t1 → 1

sub $s1, $s0, $t2 → 2

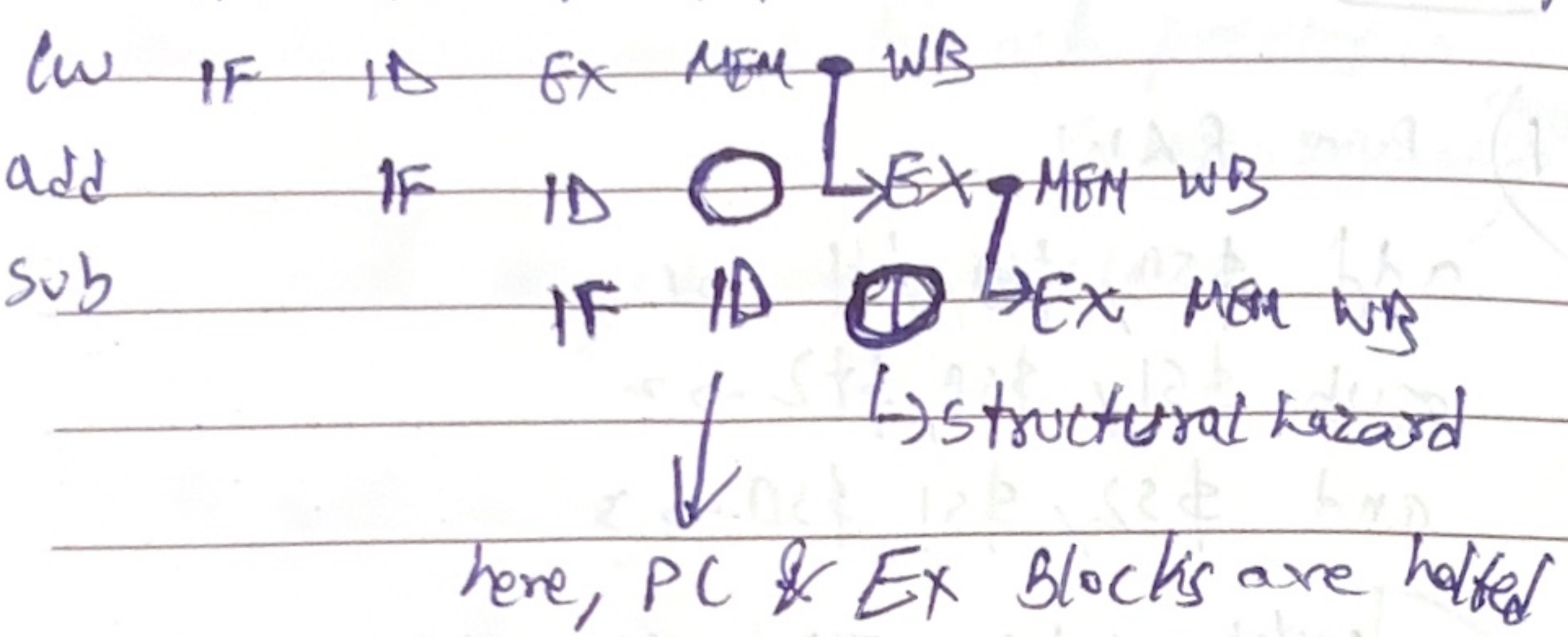
and $s2, $s1, $s0 → 3



3 forwarding paths are used and no stalls.

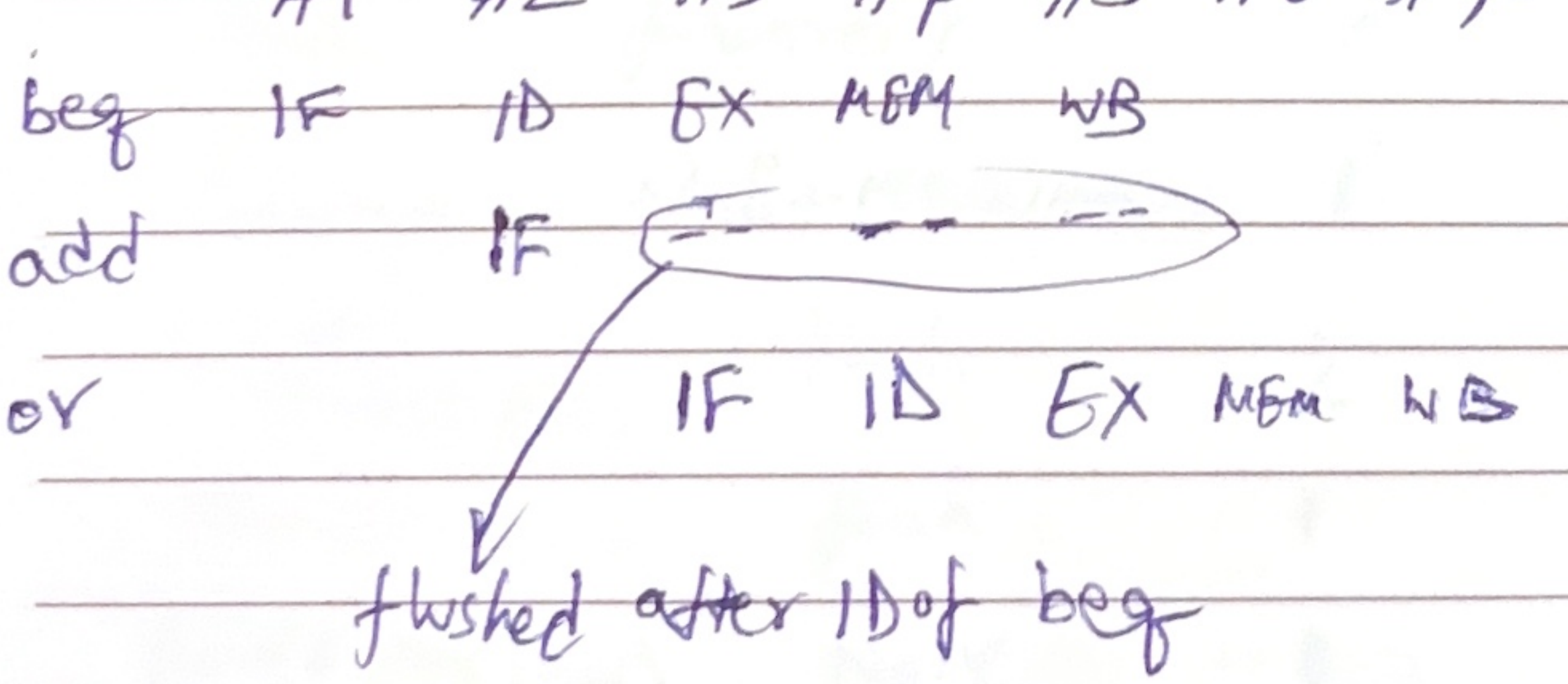
2) load-use (assume instruction & Data memory are separate)

```
lw $s0, 0($t0)
add $s1, $s0, $t1
sub $s2, $s1, $t2
```



3) control with a taken branch

```
beq $t0, $t1, target
add $s0, $s1, $s2
sub $s3, $s4, $s5
target:
or $s6, $s7, $t0
```



Date

Branch penalty is 1 cycle
and Only 1 instruction is flushed.

Part-B :

1) structural hazard : 2 instructions needing same hardware in the same cycle.

ex → in sequence 2 of Part A, for add-sub 'EX' element.

Data hazard : An instruction needing a value that isn't written yet.

ex → add $s0, $t0, $t1
sub $t2, $s0, $t3

control hazard : An issue in fetch because of branch resolution.

ex → beq

2) min. CPI for N-independent add instructions
= ~~1~~ 1 (ideal)

for N lw instructions, CPI = 1 + ~~stalls/inst~~ 1
= 2 //

Date

3) IRON Law ⇒ CPU time = CPI × IC × T_{Cycle}

⇒ IPC = $\frac{IC \times T_{cycle}}{CPU\ time}$

Each instr. takes 1 cycle to execute. and there's only a single datapath.

So, max. IPC = 1.

4) New branch penalty is only 1 cycle. (for misprediction)

We need a comparator in the ID stage that checks with the PC output.

5) RAW without a stall (forwarding)

add $s0, $t1, $t2

add $s1, $s0, $t3

add $s2, $t4, $t5

RAW with a stall (forwarding)

```
lw  $s0, 0($t0)
sub $s1, $s0, $t2
```

6) Write happens in WB and in in-order WB of later instr. can't happen before WB of present instr.

This ordering makes sure WAW & WAR can't happen. But in out-of-order they can!